A.Tejaswini-192471043

Q101. Multi-Cloud Cost Optimization for Microservices (DP + Greedy Hybrid)An enterprise runs microservices across multiple cloud providers and must minimizemonthly cost while meeting latency SLAs. Analyze how service placement can be modeledusing a hybrid of Dynamic Programming and Greedy selection. Identify constraints such asregion latency, pricing tiers, and inter-service communication costs. Design a solution thatselect optimal regions and instance types per service. Discuss strategies for handlingdynamic workloads and spot pricing. Evaluate time and space complexity of the optimizer.Interpret results in terms of cost savings and SLA compliance.

Ans: #Q101. Multi-Cloud Cost Optimization for Microservices

DP + Greedy Hybrid Approach

```
services = [
 {
 "name": "AuthService",
 "cpu": 2,
 "memory": 4,
 "sla_latency": 100
 },
 {
 "name": "PaymentService",
 "cpu": 4,
 "memory": 8,
 "sla_latency": 80
 },
 {
 "name": "NotificationService",
 "cpu": 1,
 "memory": 2,
 "sla_latency": 150
 }
]
```

```
cloud_options = [
 {
 "provider": "AWS",
 "region": "us-east-1",
 "instance": "t3.medium",
 "cpu": 2,
 "memory": 4,
 "cost": 40,
 "latency": 70
 },
 {
 "provider": "AWS",
 "region": "us-west-1",
 "instance": "t3.large",
 "cpu": 4,
 "memory": 8,
 "cost": 80,
 "latency": 90
 },
 {
 "provider": "Azure",
 "region": "east-us",
 "instance": "B2s",
 "cpu": 2,
 "memory": 4,
 "cost": 35,
 "latency": 85
 },
 {
 "provider": "Azure",
```

```
 "region": "west-us",
 "instance": "D4s",
 "cpu": 4,
 "memory": 8,
 "cost": 75,
 "latency": 75
 },
 {
 "provider": "GCP",
 "region": "us-central1",
 "instance": "e2-medium",
 "cpu": 2,
 "memory": 4,
 "cost": 30,
 "latency": 95
 },
 {
 "provider": "GCP",
 "region": "us-east1",
 "instance": "n2-standard-4",
 "cpu": 4,
 "memory": 8,
 "cost": 70,
 "latency": 65
 }
]
def optimize_microservices(services, cloud_options):
 final_placement = []
 total_cost = 0
```

```
for service in services:
valid_options = []

for option in cloud_options:
if (
option["cpu"] >= service["cpu"]
and option["memory"] >= service["memory"]
and option["latency"] <= service["sla_latency"]
):
valid_options.append(option)

if not valid_options:
print("No valid placement found for", service["name"])
continue

# Greedy selection: choose the lowest-cost valid option
best_option = min(valid_options, key=lambda x: x["cost"])

final_placement.append({
"service": service["name"],
"provider": best_option["provider"],
"region": best_option["region"],
"instance": best_option["instance"],
"cost": best_option["cost"],
"latency": best_option["latency"]
})

total_cost += best_option["cost"]

return final_placement, total_cost
```

```
placement, cost = optimize_microservices(services, cloud_options)


print("Optimal Service Placement:")
for item in placement:
 print(item)


print ("\nTotal Monthly Cost:", cost)
```

Q102. Edge Computing Task Offloading Optimization (Graph + Shortest Path)An IoT platform must decide whether to process tasks locally or offload to edge servers tominimize delay and energy. Analyze how this can be modeled as a graph with weightededges for latency and energy. Identify constraints such as bandwidth limits and devicebattery. Design a shortest-path–based offloading strategy. Discuss real-time adaptabilityunder network fluctuations. Evaluate computational overhead of decision-making. Interpretresults in reduced latency and energy consumption.

Ans: #Q102. Edge Computing Task Offloading Optimization

```
# Graph + Shortest Path using Dijkstra Algorithm


import heapq



def dijkstra(graph, start, end):
 priority_queue = []
 heapq.heappush(priority_queue, (0, start))


 distances = {}
 previous = {}


 for node in graph:
 distances[node] = float("inf")
```

```
previous[node] = None

distances[start] = 0

while priority_queue:
current_cost, current_node = heapq.heappop(priority_queue)

if current_node == end:
break

if current_cost > distances[current_node]:
continue

for neighbor, weight in graph[current_node]:
new_cost = current_cost + weight

if new_cost < distances[neighbor]:
distances[neighbor] = new_cost
previous[neighbor] = current_node
heapq.heappush(priority_queue, (new_cost, neighbor))

path = []
node = end

while node is not None:
path.append(node)
node = previous[node]

path.reverse()
```

```
 return distances[end], path


graph = {
 "IoT_Device": [
 ("Local_Process", 8),
 ("Edge_Server_1", 4),
 ("Edge_Server_2", 6)
 ],
 "Edge_Server_1": [
 ("Cloud_Server", 5),
 ("Result", 3)
 ],
 "Edge_Server_2": [
 ("Cloud_Server", 3),
 ("Result", 4)
 ],
 "Cloud_Server": [
 ("Result", 6)
 ],
 "Local_Process": [
 ("Result", 7)
 ],
 "Result": []
}

cost, path = dijkstra(graph, "IoT_Device", "Result")

print("Minimum Delay/Energy Cost:", cost)
print("Best Offloading Path:", " -> ".join(path))
```